i have meta quest 3, where i made a small youtube iframe application to collect youtube player metrics, at the same time i am collecting network metrics from smartphone which gives hotspot connection to metaquest, i collected rsrp, rsrq and snr. Now I also need energy consumption metrics during 360 streaming, how do I collect those?

Worked for 1m 5s

Yes. In your setup, I would collect **energy at two points**: (1) the Meta Quest 3 itself and (2) the smartphone acting as the 5G/Wi-Fi hotspot. Then you can synchronize both with your existing YouTube player and RSRP/RSRQ/SNR logs.

1. Meta Quest 3: use battery current + voltage

For the Quest 3, the most useful energy metrics are:

Metric	Unit	Purpose
Battery voltage	mV	Battery operating voltage
Battery current	µA or mA	Instantaneous current draw
Power	W	Instantaneous energy consumption rate
Energy	J or Wh	Cumulative consumption
Battery level	%	Validation
Battery temperature	°C	Thermal effects

The calculations are straightforward:

$$P(t) = V(t) \times |I(t)|$$

With voltage in volts and current in amperes.

Then:

$$E = \int P(t)dt$$

For 1-second samples:

$$E_J \approx \sum_i P_i \times 1$$

and:

$$E_{Wh} = \frac{E_J}{3600}$$

Android's standard `BatteryManager` exposes instantaneous current, average current, charge counter, and—when supported by the device—an energy counter.

Android Develop...

2. There is a particularly interesting way to get this from Quest 3

Your Quest is already an Android-based device, and Meta officially supports retrieving system logs through ADB/Logcat over **USB or Wi-Fi**. Meta for Develop... +1

Quest logs contain `healthd` battery records such as:

```
healthd: battery l=81 v=4073 t=41.0 h=2 st=3 c=-666886 fc=4892000 cc=31
```

Recent Quest 3 logs show exactly this format, including current and voltage.

Meta Community ... +1

The important fields appear to be approximately:

```
l  = battery level (%)
v  = voltage (mV)
t  = temperature (°C)
c  = battery current (µA)
fc = full-charge capacity (µAh)
cc = charge-cycle count
```

For the example above:

```
Voltage = 4073 mV = 4.073 V
Current = −666886 µA = −0.666886 A
```

Thus:

$$P = 4.073 \times 0.666886$$

≈ **2.72 W**.

The negative sign indicates battery discharge; for consumption you generally use the magnitude.

3. Try this first on your Quest 3

Connect your Quest through ADB and run:

```bash
adb logcat -v epoch -s healthd
```

You should see lines containing something similar to:

```
1756200000.123  0  0 W healthd: battery l=76 v=4018 t=36.0 h=2 st=3 c=-1354000
```

The `-v epoch` part is especially useful for your research because it gives you a Unix timestamp.

You could therefore create a CSV like:

```
timestamp,battery_pct,voltage_mV,current_uA,power_W,temp_C
1756200000.123,76,4018,-1354000,5.440,36.0
1756200001.207,76,4014,-1421000,5.704,36.0
...
```

This would integrate extremely well with your existing dataset.

4. Use ADB over Wi-Fi, not USB

This is important.

Do not leave the USB cable connected during your experiment, because the headset may charge from USB. Then `current` becomes **net battery current**, and you are no longer measuring pure headset battery consumption.

Meta officially supports ADB over Wi-Fi, including through Meta Quest Developer Hub. The computer and headset simply need to be on the same network; MQDH has an **ADB over Wi-Fi** option. ∞ Meta for Develop... +1

In your topology this can be:

```
                Samsung S25+
              5G → Wi-Fi Hotspot
                   /      \
                  /        \
            Quest 3        Laptop
               |             |
          YouTube 360°   ADB over Wi-Fi
               |             |
         iframe metrics   healthd metrics
```

That is probably the cleanest architecture for your experiment.

5. There is one Quest OS limitation to be aware of

You may encounter advice online suggesting:

Bash

```
adb shell cat /sys/class/power_supply/battery/current_now
```

I would **not build your experiment around that**.

On newer Quest OS versions, direct access to `current_now` under `/sys/class/power_supply/` can be permission-restricted. A recently released Quest profiling tool specifically reports this behavior on newer Quest OS builds. PyPI

So test in this order:

```
1. healthd through adb logcat      ← preferred
2. Android BatteryManager APK     ← alternative
3. sysfs current_now              ← only if accessible
4. Battery percentage difference  ← fallback
```

6. Your sampling rates do not have to be identical

Your existing data are approximately:

```
t
│
├── YouTube iframe
│       resolution
│       buffered %
│       bytes
│       quality
│       playback state
```

```
│      ...
│
├── Smartphone
│      RSRP
│      RSRQ
│      SNR
│      DL bitrate
│      UL bitrate
│      ...
│
└── Quest 3
       current
       voltage
       power
       temperature
       battery %
```

If `healthd` provides battery measurements irregularly—for example every 1–10 seconds—**do not manufacture fake 1-second measurements**.

Keep the raw timestamps first.

Then, during preprocessing, you can align everything to 1-second bins:

```
timestamp    RSRP  RSRQ SNR resolution buffering power_W
14:02:01     −104   −11  19   2160p       0       5.41
14:02:02     −105   −12  18   2160p       0       5.58
14:02:03     −108   −14  14   1440p       1       5.26
...
```

This becomes a **very interesting cross-layer dataset**.

7. Also collect Meta's performance metrics

I would add another Quest stream:

Bash

```
adb logcat -v epoch -s VrApi,XrPerformanceManager
```

Meta states that the VrApi performance log normally outputs approximately once per second and contains metrics such as:

- FPS
- refresh rate
- CPU level

- GPU level
- CPU frequency
- GPU frequency
- GPU utilization
- CPU utilization
- memory
- thermal information
- application GPU time
- compositor information. ∞ Meta for Develop...

OVR Metrics Tool can also export Quest performance sessions to CSV. ∞ Meta for Develop...

This would let you study something substantially richer:

$$\text{Network conditions} \rightarrow \text{Streaming behavior} \rightarrow \text{VR computational load} \rightarrow \text{Ene}$$

For example:

```
RSRP / RSRQ / SNR
        ↓
video bitrate
        ↓
resolution / adaptation
        ↓
decoder + GPU/CPU workload
        ↓
Quest instantaneous power
```

8. The smartphone should also have an energy logger

Because your phone is functioning simultaneously as:

```
5G UE
+
Wi-Fi hotspot
+
network measurement device
```

its power consumption is scientifically important too.

On the Samsung phone, Android's `BatteryManager` provides:

‹/› Java

```
BATTERY_PROPERTY_CURRENT_NOW
BATTERY_PROPERTY_CURRENT_AVERAGE
BATTERY_PROPERTY_CHARGE_COUNTER
BATTERY_PROPERTY_ENERGY_COUNTER
```

where supported. Android Develop...

I would make a **very small Android logger** that records once per second:

```
timestamp
battery_pct
voltage_mV
current_uA
current_average_uA
charge_counter_uAh
temperature_C
```

alongside your G-NetTrack measurements.

Then calculate:

$$P_{phone}(t) = V_{phone}(t)|I_{phone}(t)|$$

and separately:

$$P_{Quest}(t) = V_{Quest}(t)|I_{Quest}(t)|$$

This gives you:

$$P_{system}(t) = P_{phone}(t) + P_{Quest}(t)$$

although I recommend **reporting the two separately as well**, because the smartphone and HMD represent different parts of the streaming system.

9. Your final dataset could become significantly stronger

I would extend your experiment CSV conceptually to something like:

```
Experiment_ID
Timestamp

# Network
RSRP
RSRQ
SNR
Neighbor_RSRP
```

```
Neighbor_RSRQ
Neighbor_SNR
DL_bitrate
UL_bitrate

# YouTube
Resolution
Video_Bitrate
Buffer_Level
Buffering_State
Loaded_Percent
Video_Bytes
Quality_Change

# Quest performance
FPS
GPU_Utilization
CPU_Utilization
GPU_Frequency
CPU_Frequency
Quest_Temperature

# Quest energy
Quest_Current_mA
Quest_Voltage_V
Quest_Power_W
Quest_Energy_J

# Smartphone energy
Phone_Current_mA
Phone_Voltage_V
Phone_Power_W
Phone_Energy_J
```

Then you could investigate questions that go beyond your existing QoE prediction, such as:

> Does poor 5G signal quality increase end-to-end energy cost of immersive 360

and:

> What is the energy cost of maintaining 2160p versus adaptive lower-resolutio

You could even define an efficiency metric such as:

$$EE = \frac{\text{Video data delivered (Mbit)}}{\text{Energy consumed (J)}}$$

or:

$$E_{minute} = \frac{E_J}{\text{streaming duration in minutes}}$$

and compare good-QoE versus degraded-QoE sessions.

What I recommend for your exact setup

I would **not buy hardware initially**. First try:

Quest 3: `healthd` Logcat over wireless ADB → voltage + current → power → energy.

Quest performance: VrApi/OVR Metrics → CPU/GPU/FPS/thermal.

Samsung hotspot: a simple Android `BatteryManager` logger → voltage + current → power → energy.

Existing measurements: keep your YouTube iframe and G-NetTrack pipeline exactly as it is.

Afterward synchronize everything using Unix timestamps.

For publication-quality experiments, I would also standardize starting battery SOC (for example 70–80%), brightness, refresh rate, volume, passthrough state, video, video duration, and allow thermal cooldown between runs. Meta's own performance guidance emphasizes controlling CPU/GPU/performance conditions when comparing measurements. ∞ Meta for Develop…

If you want, I can next give you a **ready-to-run Python program that connects wirelessly to the Quest 3, extracts `healthd` continuously, calculates `current_mA`, `voltage_V`, `power_W`, and cumulative `energy_J`, and saves everything to CSV with timestamps matching your existing 1-second dataset**.

Sources

ChatGPT can make mistakes. Check important info.